University of Exeter

Lecture 12 – Swarm Intelligence, Part III: Particle Swarm Optimisation

ECM3412/ECMM409 – Nature-Inspired Computation

Dr David Walker

Department of Computer Science

D.J.Walker2@exeter.ac.uk

Autumn 2025

Particle Swarm Optimisation (1995)

- **Particle** – each particle is a candidate solution
  - with no mass or volume
  - but velocity and acceleration apply
- **Swarm** – velocity matching not included
- **Optimisation** – discovery of near optimal solutions by means of a population of individuals

The "Cornfield Vector" or Rooster Effect

- How does swarming or flocking get us near to an optimal solution?
- When food is left for birds there are often a number of birds at that food source within minutes or hours (similar to our optimal solution)
- Therefore we can define a fitness function which represents the quality of a solution (the odour of the food)
- The knowledge of the swarm is incorporated into the local behaviour of each particle – all collaborate to find the food quickly

Continuous optimisation problems

Often easy to think about **discrete** optimisation problems – e.g.

- **Binary** problems (such as feature selection in machine learning)

$$\mathbf{x} = (1, 1, 0, 0, 1, 0, 0, 1)$$

- **Permutation** problems (such as travelling salesman problem)

$$\mathbf{x} = (2, 0, 6, 7, 5, 1, 4, 3)$$

PSO was developed for **continuous** representations, e.g.:

$$\mathbf{x} = (-1.3220, 0.1302, 2.3125, -0.6435, -0.6435)$$

with numerous applications (e.g. the **design** of antennae, aircraft wings, amplifiers, controllers, circuits, etc. . .)

Basic PSO

Particle Swarm Optimisation

1. Create and initialise N random particles on the search space
2. For each timestep:
   1. For each individual:
      1. Update the position of the particle by adding a velocity to the current position of the particle
      2. Update the velocity

How to determine the new position $\mathbf{x}_i$ for a particle?

- Use its velocity $\mathbf{v}_i$ as follows:

$$x_{ij}^{t+1} = x_{ij}^{t} + v_{ij}^{t+1}$$

- x_{ij}^{t+1} is the position of the particle in dimension j at time $t+1$
- x_{ij}^{t} is the position of the particle in dimension j at time t
- v_{ij}^{t+1} is the particle's velocity in dimension j at time $t+1$

Updating velocities

How to determine the new position x_i for a particle?

$$v_{ij}^{t+1} = \mathbf{v_{ij}^t} + \mathbf{c_1} \times \mathbf{z_1} \times (\mathbf{p_{ij}} - \mathbf{x_{ij}}) + \mathbf{c_2} \times \mathbf{z_2} \times (\mathbf{g_j} - \mathbf{x_{ij}}),$$

The new velocity is the summation of

- The **current velocity**
- A random weighting of **the difference between the particle's best encountered position and its current position**
- A random weighting of **the difference between the population's best encountered position and its current position**

Updating velocities

$$v_{ij}^{t+1} = v_{ij}^{t} + c_1 \times z_1 \times (p_{ij} - x_{ij}) + c_2 \times z_2 \times (g_j - x_{ij}),$$

where

- c_1 and c_2 are constants – control the weighting between personal and global experience
- $z_1, z_2 \sim U(0, 1)$ are two different draws from the uniform distribution between 0 and 1 (prevents premature convergence)
- i indicates the particle and j indicates the dimension
- p_{ij} is the particle's best position (on dimension j) – draws a particle back towards the fittest areas of the landscape they have encountered
- g_j is the population's best position (on dimension j) – enables the experience of the whole swarm to guide the particle

PSO settings

Parameters

- **Swarm size** – analogous to population size in GAs
- **Neighbourhood size** – for calculating **local best**
- Number of **iterations**
- Acceleration coefficients $\mathbf{c_1}$ and $\mathbf{c_2}$
  - To manage the relationship between **pbest** and **gbest**/**lbest**
  - Too low values can slow progress
  - Too high values may cause premature convergence

Termination criteria

- Maximum # iterations
- Acceptable solution is found
- No improvement for N iterations
- Normalised swarm radius close to zero (**converged**)

Binary PSO

PSO generally requires a real vector representation but can be modified to be modified to work on discrete spaces (e.g., binary strings)

Approach 1: interpret velocities as probabilities (traditional BPSO)

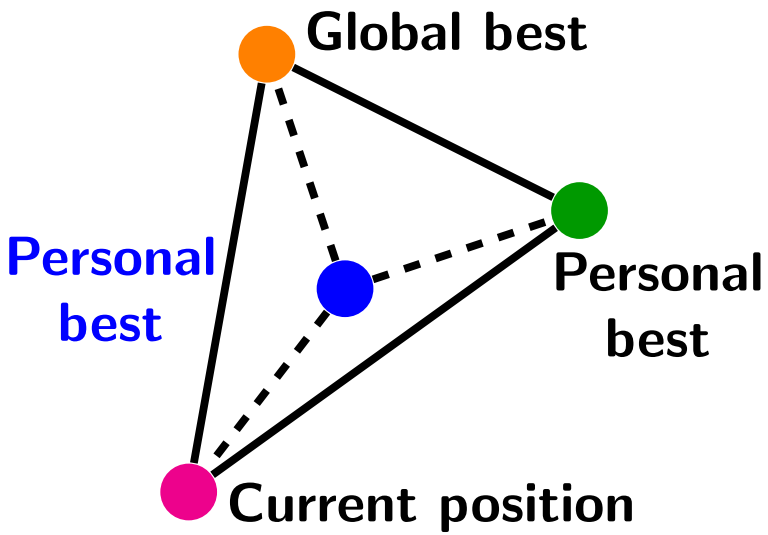
- Velocity remains continuous using the original update rule
- Positions are updated using the velocity as a probability threshold to determine if the j-th component of the i-th particle is a 0 or a 1:

$$x_{ij}^{t+1} = \begin{cases} 1 \text{ if } \text{rand}() < v_{ij}^{t} \\ 0 \text{ otherwise} \end{cases}$$

Binary PSO

Approach 2: generalise motion to discrete spaces (geometric BPSO)

No explicit velocity – positions are updated by moving the particle towards personal best and global best proportionally to $\mathbf{c_1}$ and $\mathbf{c_2}$



In the Hamming space this corresponds to a multi-string recombination

```
Mask : (1 2 3 2 2 1 3)

Cpos : (0 1 0 1 1 1 1)
Gbest: (1 1 0 1 0 0 1)
Pbest: (1 0 0 0 1 0 0)

Npos : (0 1 0 1 0 1 0)
```

Summary

Particle Swarm Optimisation

- Loosely based on the principles of swarming
- Has a simple formula to update the velocities of particles and therefore create new solutions
- Has an intuitive set of parameters
- Can be modified to be used on discrete spaces and dynamic optimisation problems

Swarm Intelligence

- Collectives of simple individuals can generate intelligent behaviours that are useful in creating search algorithms
- The mechanisms by which they communicate can vary considerably – **stigmergy** in ACO and **Gbest/Pbest** in PSO
- There are a number of other algorithms that use a swarm approach